



SUPHAKON SAEPHAN

FULLSTACK DEVELOPER INTERN

CONTACT

- 📞 095-218-6772
- ✉ pyaksda@gmail.com
- 🌐 <https://github.com/66JJN>
- 📄 <https://66jjn.github.io>

SKILLS

Front-End:

- JavaScript (ES6+), HTML/CSS
- React.js, TailwindCSS
- Responsive UI
- Axios, Real-time UI

Back-End:

- Node.js, Express.js, Socket.IO
- MongoDB (Mongoose)
- JWT Authentication
- Cloudinary, Multer
- Nodemailer
- Tesseract.js (OCR), Helmet
- Rate Limiting

AI/API Integration:

- Google Gemini API
- SightEngine API
- Google OAuth 2.0

Tools:

- Git, GitHub, Vercel, Render
- Postman, PlantUML
- Docker
- Antigravity / Cursor (AI-assisted dev)

EDUCATION

Bachelor of Science in Computer Science
University of Phayao
2023 – 2027 (Expected)
GPA: 3.59

OBJECTIVE

CS student (GPA 3.59) who built CMES — a real-time digital signage platform used by multiple venues. Handles image queues, AI moderation, OCR slip verification, and multi-tenant data isolation across 2 Node.js backends, 2 React frontends, and MongoDB.

PROJECT EXPERIENCE



CMES

Tech: React.js, Socket.IO, Node.js, Express, MongoDB, Cloudinary, Tesseract.js

- Built real-time image/text queue with admin approval workflow — auto-plays approved content on venue screens via Socket.IO
- Designed multi-tenant architecture using shopId-based isolation across MongoDB, Socket.IO rooms, and Express middleware
- Implemented JWT + Google OAuth authentication with Email OTP verification (Nodemailer) and role-based route protection
- Built OCR-based slip verification using Tesseract.js — extracts Thai/Arabic numerals from payment slips and validates against expected amounts
- Integrated SightEngine AI for image moderation — auto-approves safe content, flags inappropriate images (nudity, weapons, gore) for admin review
- Added AI caption generation via Gemini 2.5 Flash with retry logic and model fallback
- Built income analytics dashboard with revenue trends, peak hours, growth %, and top supporter rankings via MongoDB aggregation pipelines
- Created OBS Browser Source overlay and Lucky Wheel feature — broadcast spin events to venue screens via Socket.IO
- Hardened both backends with Helmet, rate limiting (auth/upload/AI), NoSQL injection sanitizer, and scheduled cron cleanup for old media
- Deployed split architecture: 2 frontends on Vercel, 2 backends on Render — with cross-service communication via REST



AirSafeTH

Tech: React.js, Vite, Tailwind CSS, API Integration (AQI)

- Built location-based air quality web app using React + TailwindCSS with real-time PM2.5 data from external AQI API
- Custom React hooks (useAQI, useGeolocation) for geolocation-aware data fetching with loading/error states
- Fallback to regional mock data when API returns errors or user denies location permission